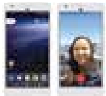
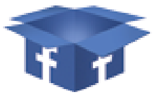


Android O Developer Preview 4
The final release before official release starts rolling out with more bug fixes and optimisations. **http://dgit.in/AndroidO4**



Facebook acquires Source3
The social media giant has reportedly acquired startup Source3. This could help content creators get paid easier. **http://dgit.in/Source3fb**

You can see how we've broken down our app into components here.

```
<github-user v-for="user in
users" :user="user"/>
```

Now let's move the search input and button into another component. Here we see the return of the good old data and methods in the context of a component. The main difference you'll see here is that data is a function that returns and object with the required properties.

Since a component can have multiple instances, a single data object would mean that all component instances will share the same data state! This is obviously not what you'd want. Using a function instead creates a new copy of your data for each instance.

We've also introduced the new $emit method here inside search. This is how your own custom components can create your own custom events. The component using this search-bar can just listen for the search event using @search without caring about how it's triggered.

You'll notice that we've added what seems like an unnecessary `div` around the input and button tags, however this is required by Vue for the component to work. A component needs to return a single element no matter what it is in the code.

Let's see what our main code looks like once we use both these components here: **http://dgit.in/Codecomp**.

Put the above code and both the component definitions in your JavaScript file and you will see the app works as expected but now it's a lot more modular.

Our main code no longer needs to store the query, instead it gets query as a parameter with the search event emitted by the search-bar. This is pretty useful

COMPUTED PROPERTIES

This is a small but clever feature of Vue that let's you create properties that are calculated from other properties. For instance you ask users for the temperature in Fahrenheit and have a computed property that automatically converts it to Celsius. Here is a minimal example of that in work:

```
Vue({
    ...
    data: {
        temp_f: 0
    },
    computed: {
        temp_c: function() {
            return (this.
temp_f - 32) / 1.8;
        }
    }
});
```

Any time you make a change to the `temp_f` property `temp_c` will automatically be updated. While this particular example only goes one way, it's possible to create computed properties that go in both directions.

PARTING WORDS

Vue.js is a great little framework that packs a huge punch in a pretty small package. It leverages your existing knowledge of HTML tags and how they work instead of having you learn yet another API for building UIs.

There is still a lot more depth to Vue than what you have seen here. For instance Vue has:

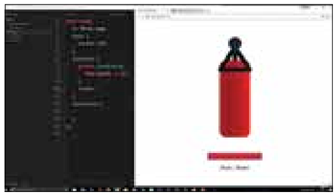
- Watcher function that run when a property is changed
- A JavaScript-based API for creating UIs
- Support for creating custom v-xyz directives
- Mixins and Plugin support
- Support for Server-side rendering

There are also a number of other addon packages available for Vue that add support for managing global state, routing and a whole lot more. If you're looking to build a brand new web app, and want a framework with a good features, and small size that's actively maintained and has a good community around it, Vue is a good bet to make. d

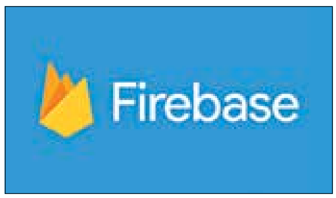
***POINTERS**

>>Interesting developer videos

***Vue Fundamentals**
This crash course from Traversy media covers all Vue.js 2.0 basics.
http://dgit.in/VueJSFun



***Simple Punchbag game**
A simple game using Vue JS 2
http://dgit.in/VuePunch



***Vue and Firebase**
Set up a Firebase realtime database and bind to real time data in your Vue application
http://dgit.in/VueFyre

***Migrating from Angular to Vue**
For a slow stack at Glovo, A new solution appeared: VueJS
http://dgit.in/Migratn